

REALTEK

Software Development Kit

Switch Product

Boot Loader U-Boot-2011.12 User Guide

(CONFIDENTIAL: Development Partners Only)

Rev. 1.3
07 June 2014



Realtek Semiconductor Corp.

No. 2, Innovation Road II, Hsinchu Science Park, Hsinchu 300, Taiwan

Tel.: +886-3-578-0211 Fax: +886-3-577-6047

www.realtek.com

COPYRIGHT

© 2008-2013 Realtek Semiconductor Corp. All rights reserved. No part of this document may be reproduced, transmitted, transcribed, stored in a retrieval system, or translated into any language in any form or by any means without the written permission of Realtek Semiconductor Corp.

TRADEMARKS

Realtek is a trademark of Realtek Semiconductor Corporation. Other names mentioned in this document are trademarks/registered trademarks of their respective owners.

DISCLAIMER

Realtek provides this document “as is”, without warranty of any kind, neither expressed nor implied, including, but not limited to, the particular purpose. Realtek may make improvements and/or changes in this document or in the product described in this document at any time. This document could include technical inaccuracies or typographical errors.

USING THIS DOCUMENT

This document is intended for use by the system engineer when integrating with Realtek Switch Software SDK. Though every effort has been made to assure that this document is current and accurate, more information may have become available subsequent to the production of this guide. In that event, please contact your Realtek representative for additional information that may help in the development process.

REVISION HISTORY

Revision	Release Date	Summary
1.0	2013/01/16	Init draft
1.1	2013/03/29	Add section 4.1.1 pll_mhz structure description.
1.2	2014/01/08	Modified the u-boot image size from 0x40000 to 0x80000 in section 5.2.
1.3	2014/06/07	Modify vmlinux.bix boot up address.

Table of Contents

Chapter 1	Introduction	4
1.1.	Features	4
Chapter 2	Environment Setting	5
2.1.	toolkit_path.in	5
Chapter 3	Loader Code Flow	6
3.1.	Introduction	6
3.2.	Boot up Main Flow	6
3.3.	c_start() Code Flow - SoC parts	8
3.4.	flash_init() - SoC parts	10
3.5.	misc_init_r() - Switch parts	10
Chapter 4	DDR2/3 & SPI flash Data Bases	12
4.1.	soc_init_parameters.c	12
4.1.1.	pll_mhz data structure	12
4.1.2.	soc_in_flash data structure	12
4.1.3.	dram_info data structure	13
4.1.4.	flash_info data structure	13
4.2.	DDR2/3	14
4.3.	SPI Flash	14
Chapter 5	Updated Image Procedures	15
5.1.	Updated Runtime (vmlinux.bix) Image	15
5.2.	Updated U-Boot (u-boot.bin) Image	15

Chapter 1 Introduction

1.1. Features

- Support SPI flash auto probe from pre-created data base.
- To assort the suitable hardware design, the on-board DDR2/3 chip could be auto-selected.

Chapter 2 Environment Setting

2.1. `toolkit_path.in`

In the `toolkit_path.in`, the parameter **CROSS_COMPILE** defines the cross compile prefix, such as "msdk-linux-" or "mips-linux-uclibc-".

If the Makefile of `./u-boot-2011.12` folder will be called from other Makefile and the **CROSS_COMPILE** parameter is specified from outside, it will be replaced. Due to the **CROSS_COMPILE** is assigned by "=?",

The generated boot loader image "**u-boot.bin**" will be stored in `./u-boot-2011.12/` folder and `/image/` folder.

Chapter 3 Loader Code Flow

3.1. Introduction

Basically, the code flow design is following the official U-boot architecture.

There are two major parts are added and modified into U-Boot-2011.12.

- (1) SoC parts (includes CPU init, DDR2/3 driver, Flash driver, console...etc)
- (2) Switch parts (includes MAC init and PHY init...etc)

3.2. Boot up Main Flow

Please refer the Figure 3.2-1, the brief whole boot up flow chart is shown.

- CPU initialization is done by assemble code in SoC parts in Figure 3.2-1.
- `c_start()` and `flash_init()` initialize all SoC peripherals without CPU part.
- `misc_init_r()` initializes the switch part.

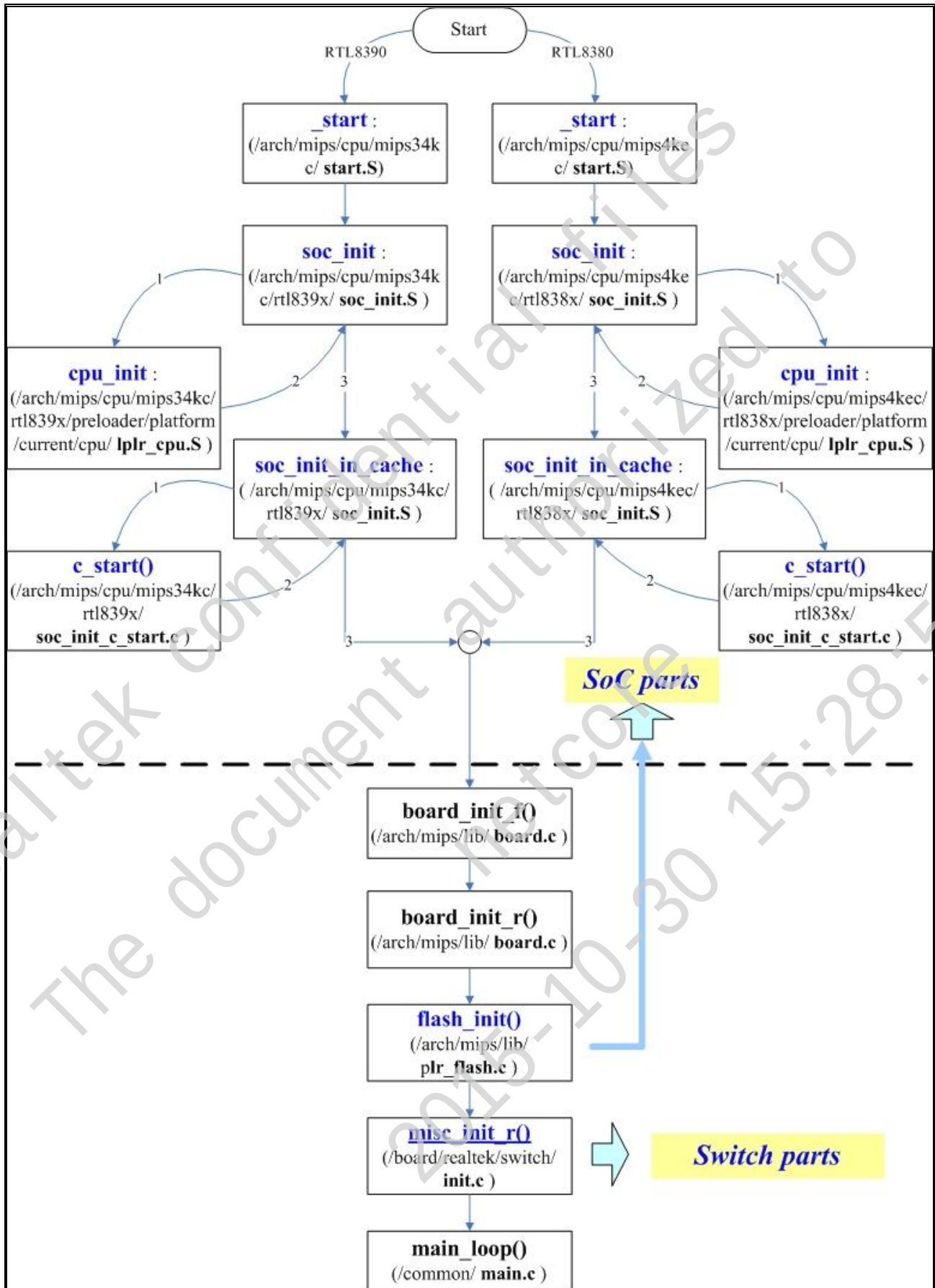


Figure 3.2-1. boot up main flow chart

3.3. `c_start()` Code Flow - SoC parts

In SoC parts, the `c_start()` is used to initialize the peripherals without CPU and flash driver, such as UART, PLL and DDR2/3 initialization and DDR2/3 calibration algorithm.

The `c_start()` flow chart is shown in Figure 3.3-1.

- `parameters_setup()` : This API is used to get the SoC related parameters and fill them into the global data structure which will be used by later APIs.
- `console_init()` : This API will initialize the UART peripheral and hook console driver's APIs.
- `pll_setup()` : Set up the PLL peripheral.
- DDR2/3 parts :
 - `dram_model_select()` : Return the index of DDR data base, and this DDR chip will be used in device.
 - `get_dram_spare()` : Get the DDR2/3 chip related parameters from data base by index.
 - `dram_setup()` : Set the DDR parts of memory controller and execute the DDR2/3 calibration algorithm.

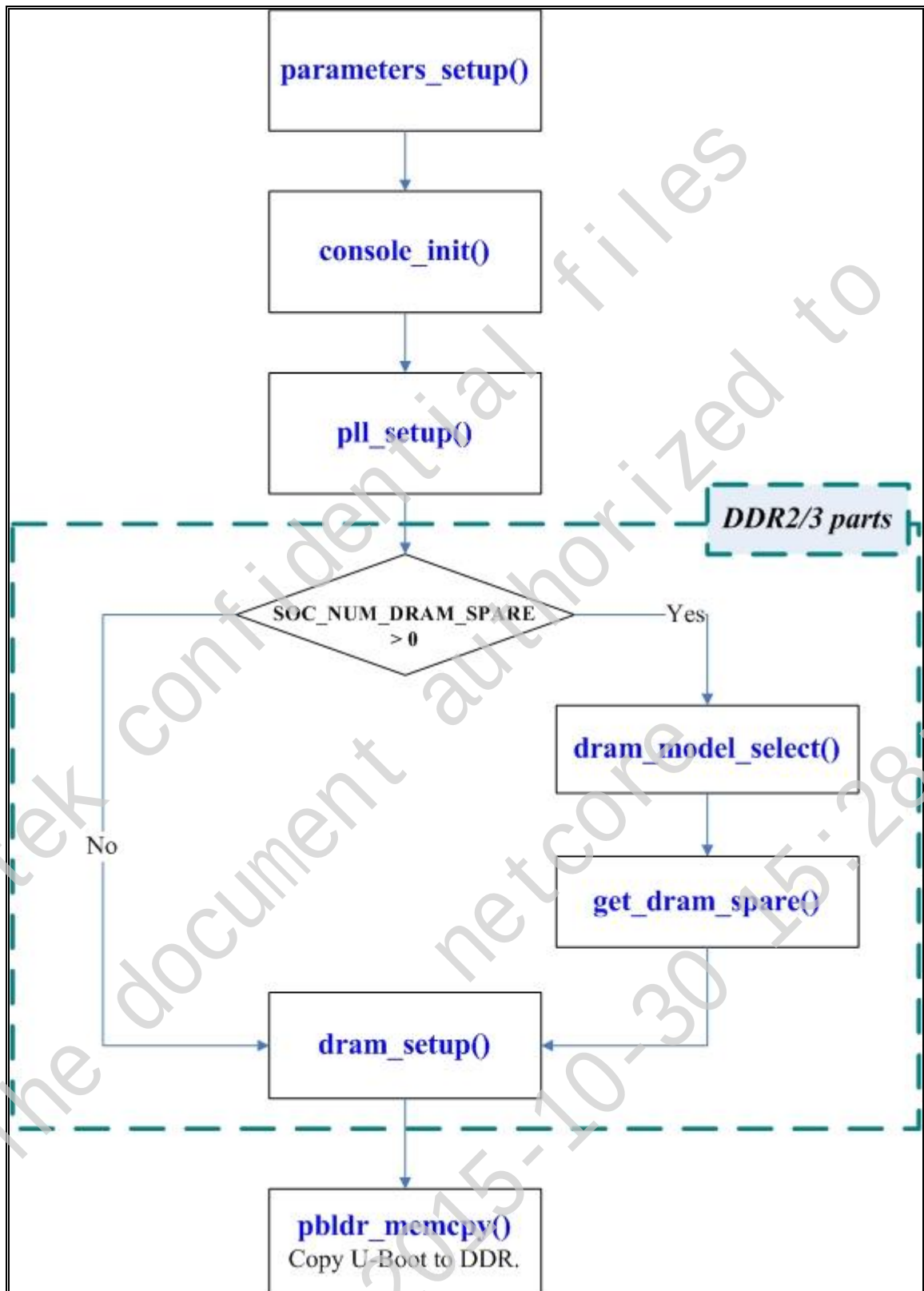


Figure 3.3-1. The c_start() flow chart

3.4. flash_init() - SoC parts

The Figure 3.2-1 shows the APIs' are called sequence and the flash_init() API is called in the bottom half of U-Boot-2011.12 image.

This API has two major purposes,

- (1) Probe the SPI flash chip ID(RDID) and use this chip ID to search the existed SPI flash data base. Once this chip is supported in the existed data base, then the related parameters will be used to configure memory controller.
- (2) Hook up the SPI flash driver's APIs.

3.5. misc_init_r() - Switch parts

The Figure 3.5-1 shows the misc_init_r() code flow.

- board_probe() : This API will extract the "board_model" parameters from U-boot environment parameters. Once the parameter is existed, then this parameter will be used to search existed board data base.
This board module data structure includes MAC, PHY and LED combination.
- mac_drv_init() : Hook the MDC/MDIO driver's APIs.
- Chip_config() : MAC initialization, PHY initialization and patch code, Serdes initialization and patch code are executed in this API.

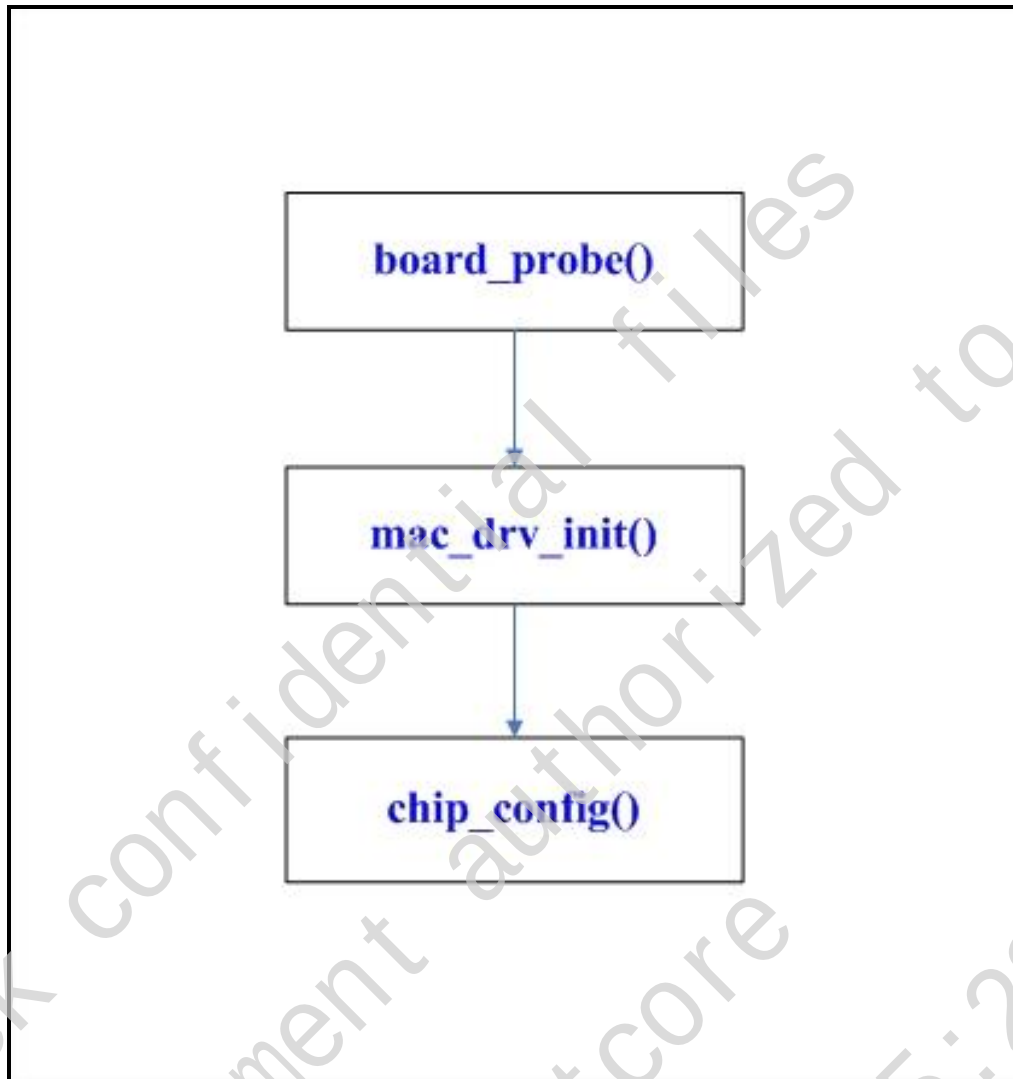


Figure 3.5-1. The `misc_init_r()` flow chart

Chapter 4 DDR2/3 & SPI flash Data Bases

4.1. soc_init_parameters.c

This file stores all SoC related parameters and data bases, such as DDR2/3, Flash and PLL...etc.

4.1.1. pll_mhz data structure

The following is an example for PLL configuration. The parameter ".cpu" is used to CPU clock. The parameter ".mem" is used to DDR2/3 bus clock.

```
pll_gen2_mhz_t pll_mhz = {
    .mode = PLL_MODE_BY_SW,
    .cpu = 750,
    .lx = 200,
    .mem = 400,
};
```

4.1.2. soc_in_flash data structure

The following is an example in this file. The **soc_in_flash** is the most important data structure which includes all SoC related parameters. By default, all SoC parameters will be fetched from here at first.

```
const soc_t soc_in_flash = {
    .header_ver = SOC_HEADER_VERSION,
    .
    .
    .
    .flash_info = {
        #include <NOR_SPI_F_MXIC_MX25L12835F.h>
    },
    .
    .
    .
```

```

.dram_info = {
#ifdef CONFIG_CUSTOMER_BOARD
    /* Realtek's default parameters of DRAM. */
    #include <DRAM_GEN2_SAMSUNG_K4B1G0846G_300MHZ.h>
#else
    /* Customer's parameters of DRAM. */
    #include <DRAM_CUSTOM_MODEL.h>
#endif
},
.
.
.
}

```

4.1.3. **.dram_info** data structure

The **.dram_info** includes the specific DDR2/3 chip parameters and DDR data base index 0 is located in this **soc_in_flash** structure. These index 1 and later are located in **dram_info_in_spare**.

```

const dram_info_t dram_info_in_spare[SOC_NUM_DRAM_SPARE]
= {
    {
        #include <DRAM_GEN2_SAMSUNG_K4B1G0846G_300MHZ.h>
    }, {
        #include <DRAM_GEN2_SAMSUNG_K4B1G0846G_300MHZ.h>
    }
};

```

4.1.4. **.flash_info** data structure

The **.flash_info** includes the specific SPI flash chip parameters and flash data base index 0 is located in this **soc_in_flash** structure. These index 1 and later are located in **dram_info_in_spare**.

```
const flash_info_t
flash_info_in_spare[SOC_NUM_FLASH_SPARE] = {
    {
        #include <NOR_SPI_F_EON_EN25Q64.h>      /*index 1*/
    }, {
        #include <NOR_SPI_F_MXIC_MX25L25635E_3B_NQIO.h>
        /*index 2*/
    }
};
```

4.2. DDR2/3

The SOC_NUM_DRAM_SPARE is stored soc.h and this macro is used to index how many supported DDR2/3 chips in the **dram_info_in_spare** data base.

DDR2/3 cannot be auto-detected. If the SOC_NUM_DRAM_SPARE is zero, then **.flash_info** will be used. Once the SOC_NUM_DRAM_SPARE and dram_model_select() return value are larger than 0 both, DDR2/3 chip related parameters will be fetched from **dram_info_in_spare** data base.

The dram_model_select() API could be redesigned by customer. Maybe customer can detect GPIO as stripping pin to decide which DDR chip is used in this device.

4.3. SPI Flash

The SOC_NUM_FLASH_SPARE is stored soc.h and this macro is used to index how many supported SPI flash chips in the **flash_info_in_spare** data base.

The SPI flash could be auto-probed, so when the SPI flash RDID is get, then the RDID will be used as index to find suitable chip parameters from **.flash_info** and **flash_info_in_spare**.

Chapter 5 Updated Image Procedures

5.1. Updated Runtime (vmlinux.bix) Image

The vmlinux.bix image could be loaded into device by "tftp" **or** "kermit"(loadb).

- 1) RTL839x# loadb 0x81000000 [**select** the "vmlinux.bix", then **send**]
or
- 1) RTL839x# tftp 0x81000000 vmlinux.bix
- 2) RTL839x# sf probe 0
- 3) RTL839x# sf erase 0x2a0000 0x300000 (depend on image size)
- 4) RTL839x# sf write 0x81000000 0x2a0000 0x227a51 (depend on image size).
- 5) reboot the device ... (Power-off, then power-on)
- 6) setenv bootcmd 'rtk network on; bootm 0xb42a0000'
- 7) saveenv
- 8) reboot the device again ... (Power-off, then power-on)

5.2. Updated U-Boot (u-boot.bin) Image

The loader.img image could be loaded into device by "tftp" **or** "kermit"(loadb).

- 1) RTL839x# loadb 0x81000000 [**select** the "u-boot.bin", then **send**]
or
- 1) RTL839x# tftp 0x81000000 u-boot.bin
- 2) RTL839x# sf probe 0
- 3) RTL839x# sf erase 0x0 0x80000
- 4) RTL839x# sf write 0x81000000 0x0 0x80000
- 5) reboot the device ... (Power-off, then power-on)

Realtek Semiconductor Corp.

Headquarters

No. 2, Innovation Road II

Hsinchu Science Park, Hsinchu 300, Taiwan

Tel.: +886-3-578-0211. Fax: +886-3-577-6047

www.realtek.com